VORAXIA

# Voraxia Camera Component User Manual

## Setup, runtime workflow, Blueprint integration, C++ API guidance, and debugging

| | |
|---|---|
| **Version:** | Current |
| **Date:** | 29 June 2026 |
| **Prepared By:** | Coding Custard Studios |
| **Project:** | Voraxia |
| **Status:** | User Manual |

# Contents



## Current implementation scope

- Implemented: pivot-based framing, runtime blending, pitch limits, yaw follow, yaw constraints, movement anticipation, pitch/speed modifiers, predictive collision, occlusion dithering, focus targeting, scan data queries, shoulder swapping, camera shakes, settings assets, smooth settings transitions, and Slate debug UI.
- Not part of the current build: SpringArm integration, camera animation sequences, zone priority/stacking, full spline camera mode, and asteroid-scanner gameplay logic.

# 1. System Overview

VoraxiaCamera is a manual third-person camera system designed around a **pivot point**. The system does not use **USpringArmComponent**. Instead, the camera position and rotation are calculated entirely in code from the owning actor, current rotation, framing offsets, dynamic modifiers, collision state, and temporary runtime overrides.

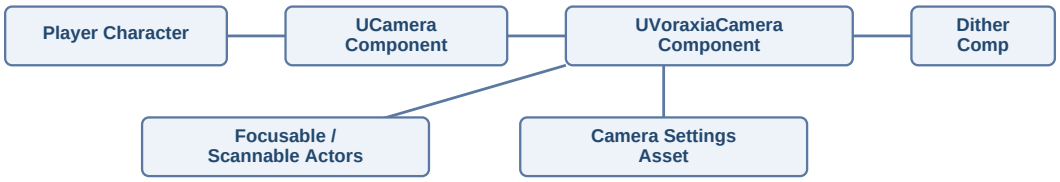
| Design principle | Manual, deterministic camera transform calculation. No SpringArm. |
|---|---|
| Primary runtime component | UVoraxiaCameraComponent |
| Optional companion component | UVoraxiaCameraOcclusionDitherComponent |
| Persistence asset | UVoraxiaCameraSettingsAsset / FVoraxiaCameraPersistentSettings |
| Main integration target | A character or pawn that owns a UCameraComponent and forwards look / action input |

## Main outcomes of this design:

- Predictable framing and orbit behaviour.
- Independent pivot offset and camera offset workflows.
- Smooth runtime blending of framing values without overwriting the authored defaults.
- Direct control over collision, predictive avoidance, focus behaviour, shoulder swapping, and preset application.

# 2. Architecture and Update Pipeline

## Component relationship




## Update pipeline

Each frame, the camera moves through a fixed conceptual pipeline. Understanding this order explains why some systems override others and why visual behaviour stays stable.


- Input and rotation logic determine the desired yaw and pitch.
- Pivot location is calculated from the owner, pivot height, and pivot offset.
- Framing determines distance, shoulder, camera offset, and FOV contributions.
- Movement anticipation, pitch modifiers, and speed modifiers apply additive changes.
- Collision and predictive avoidance compress or protect the camera path.
- The final camera transform is pushed into the target UCameraComponent.

# 3. Quick Start

## C++ setup

A typical character owns three camera-related pieces: a normal **UCameraComponent**, the main **UVoraxiaCameraComponent**, and optionally the **UVoraxiaCameraOcclusionDitherComponent**. The camera component drives the view; the Voraxia component computes the behaviour; the dither component hides blockers between the camera and the player.

```
// Character members
UPROPERTY(VisibleAnywhere)
TObjectPtr<UCameraComponent> CameraComponent;

UPROPERTY(VisibleAnywhere)
TObjectPtr<UVoraxiaCameraComponent> VoraxiaCameraComponent;

UPROPERTY(VisibleAnywhere)
TObjectPtr<UVoraxiaCameraOcclusionDitherComponent> CameraOcclusionDitherComponent;

// Constructor
CameraComponent = CreateDefaultSubobject<UCameraComponent>(TEXT("Camera"));
VoraxiaCameraComponent = CreateDefaultSubobject<UVoraxiaCameraComponent>(TEXT("VoraxiaCamera"));
CameraOcclusionDitherComponent =
CreateDefaultSubobject<UVoraxiaCameraOcclusionDitherComponent>(TEXT("CameraOcclusionDither"));

// After restart / begin play
VoraxiaCameraComponent->SetTargetCamera(CameraComponent);
CameraOcclusionDitherComponent->SetCameraComponent(VoraxiaCameraComponent);
```

## Blueprint setup

- Add a standard Camera Component to the character or pawn.
- Add the VoraxiaCamera component to the same actor.
- Optionally add VoraxiaCameraOcclusionDitherComponent for fade-through blockers.
- If Auto Find Camera Component is enabled, the camera is found automatically; otherwise call SetTargetCamera.
- Expose input actions for look, focus, shoulder swap, and any testing or preset workflow you need.

### Recommended default input actions

| Action | Usage |
|---|---|
| IA_Look | Axis2D look input forwarded into AddYawInput / AddPitchInput. |
| IA_Focus | Digital action that calls SetFocus... on press and ClearFocus on release or toggle. |
| IA_SwapShoulder | Digital action bound to SwapCameraShoulder(). |
| Optional testing action | Use a temporary action to call ApplyCameraSettingsAssetSmooth between presets. |

# 4. Core Runtime Features

| Subsystem | What it does |
|---|---|
| Framing | Base distance, pivot height, camera offset, pivot offset, and FOV offset can all be blended at runtime. |
| Shoulder camera | Dedicated Shoulder Offset and Use Right Shoulder properties replace the old AdditionalCameraOffset.Y workflow. |
| Rotation | Initial pitch, min/max pitch, look speeds, optional inversion, and optional rotation lag. |
| Yaw constraints | Temporary yaw windows with soft zones and smooth activation/release for terminals or restricted interactions. |
| Movement anticipation | Offsets the camera based on movement direction and speed to improve readability. |
| Pitch / speed modifiers | Adds dynamic distance and FOV changes based on view pitch and owner speed. |
| Camera shake | Manual and automatic shake support through handles and callbacks. |

## Runtime blending model

Most runtime changes do not overwrite the authoring values on the component. Instead they set temporary runtime targets with a blend time and optional curve. This means you can change framing during play, then return to the authored defaults safely.

## Key runtime methods

| Method | Purpose |
|---|---|
| SetFraming(...) | Blend distance, pivot height, camera offset, pivot offset, and FOV offset together. |
| ResetFraming(...) | Return the full runtime framing layer to the component defaults. |
| SetCameraDistance / ResetCameraDistance | Adjust only distance. |
| SetPivotHeight / ResetPivotHeight | Adjust only pivot height. |
| SetCameraOffset / ResetCameraOffset | Adjust the runtime camera offset layer. |
| SetPivotOffset / ResetPivotOffset | Adjust the runtime pivot offset layer. |
| SetFOVOffset / ResetFOVOffset | Adjust the runtime FOV layer. |
| SetUseRightShoulder / SetShoulderOffset / SwapCameraShoulder | Control shoulder side and magnitude. |

# 5. Focus and Scan Integration

Focus changes how the camera rotates. When focus is active, the camera steers toward an actor, component, or world location, blending in and out over time. Focus can be set directly from C++, from Blueprint, or by finding an actor with the configured DefaultFocusActorTag.

| Focus API | Purpose |
| --- | --- |
| SetFocusActor(AActor* ...) | Focus an actor, optionally using a socket name and blend time. |
| SetFocusComponent(USceneComponent * ...) | Focus a specific component or mesh component. |
| SetFocusWorldLocation(FVector ...) | Focus an arbitrary world point. |
| ClearFocus(...) | Release focus back to normal gameplay rotation. |
| FocusDefaultTaggedActor(...) | Find the first actor using DefaultFocusActorTag and focus it. |

## Blueprint integration pattern

- Use an input action such as IA_Focus.
- On Started: call a focus node (for example SetFocusActor or FocusDefaultTaggedActor).
- On Completed / Canceled: call ClearFocus if you want hold-to-focus behaviour, or use a toggle pattern if preferred.

## Scan readout integration

The camera component also exposes scan-oriented query helpers. These do not perform asteroid composition gameplay by themselves; instead they surface data from the current focused target so UI or later gameplay code can consume it.

- IsCurrentFocusTargetScannable()
- GetCurrentFocusScanDisplayName()
- GetCurrentFocusScanSummary()
- GetCurrentFocusScanTimeSeconds()
- GetCurrentFocusScanCompositionSummary()

### Expected target-side integration

A focusable target should expose focus information, and a scannable target should expose scan text and timing. In practice this is typically implemented through project interfaces (for example a focusable interface and a scannable interface) or equivalent C++ actor methods.

# 6. Collision, Predictive Avoidance, and Occlusion Dithering

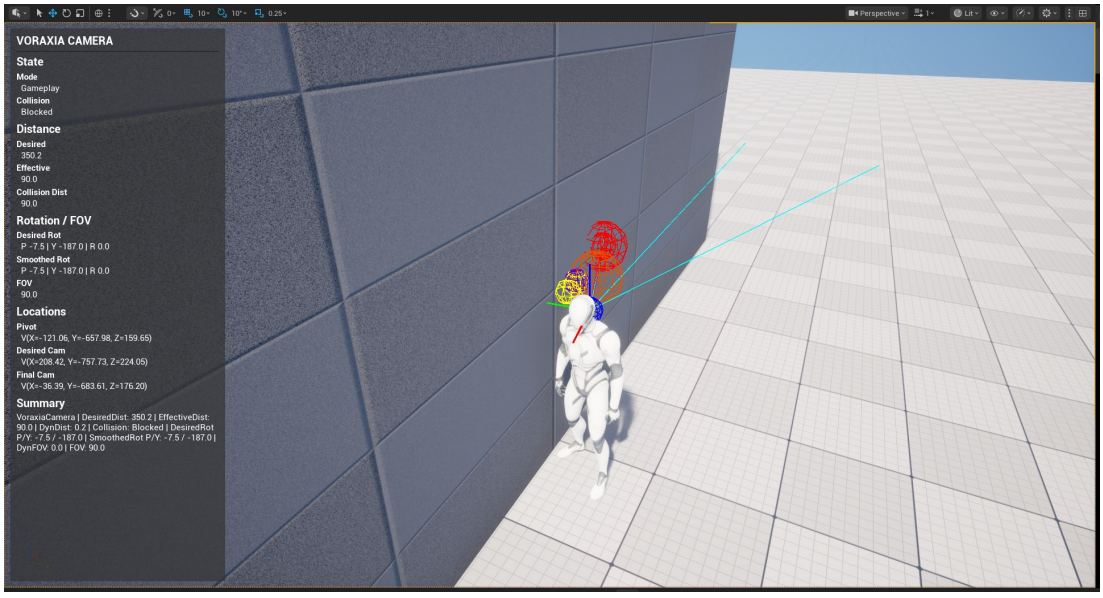
Camera collision compresses the camera toward the pivot when geometry blocks the desired camera path. Predictive avoidance adds feelers so the camera can anticipate nearby obstacles before a hard collision occurs. The occlusion dither companion component solves the separate problem of geometry that remains between the camera and the character even when the camera itself is valid.

| Collision subsystem | Key authoring values |
| --- | --- |
| Collision | CameraCollisionChannel, CameraCollisionRadius, MinDistanceFromPivot, CollisionCompressionSpeed, CollisionRecoverySpeed, CollisionSafetyPadding. |
| Predictive avoidance | bEnablePredictiveAvoidance, FeelerYawOffset, FeelerPairs, FeelerProbeRadius. |
| Dither component | DitherTraceChannel, DitherProbeRadius, DitherTraceInterval, MaxDitheredComponents, OccludedDitherFadeValue, fade in/out speeds. |

## Material setup for dithering

- Use a material that supports masked opacity.
- Add a scalar parameter named CameraDitherFade (or match whatever parameter name the dither component is configured to use).
- Feed that parameter into DitherTemporalAA and then into Opacity Mask.
- Assign the dither-ready material to any mesh you want to fade when it blocks the player.

## Debug screenshot




Example of the Slate debug panel and collision visualisation in-editor.

# 7. Camera Shakes and Callbacks

VoraxiaCamera supports manual camera shakes through stable handles. The owning local player camera manager plays the actual shake instance; the camera component keeps a safe handle so gameplay code can start, stop, or query that shake later.

| Function / event | Purpose |
| --- | --- |
| PlayCameraShake(...) | Start a shake and receive an FVoraxiaCameraShakeHandle. |
| StopCameraShake(...) | Stop one shake, immediately or with blend-out. |
| StopAllCameraShakes(...) | Stop all tracked shake instances. |
| IsCameraShakePlaying(...) | Query whether a given handle is still active. |
| OnCameraShakeStarted | Blueprint-assignable callback fired after a shake starts. |
| OnCameraShakeStopped | Blueprint-assignable callback fired after a shake ends or is interrupted. |

## Automatic shakes

The component also contains optional automatic shake hooks for idle and movement states. These are configured through component properties and are useful for subtle breathing, locomotion texture, or sprint energy without hand-coding every effect.

# 8. Settings Assets and Smooth Preset Transitions

Settings assets are the recommended way to save, reapply, and compare full camera presets. The data asset stores persistent tuning only: not the current focus target, not a live shake instance, and not the transient runtime blend state.


## Core workflow

- Tune the live camera component in-editor.
- Assign a UVoraxiaCameraSettingsAsset to Assigned Settings Asset.
- Use CaptureCurrentSettingsToAssignedAsset to persist the component state into the asset.
- Use ApplyAssignedSettingsAsset for immediate application or ApplyAssignedSettingsAssetSmooth for a visible blend at runtime.
- Use multiple assets for exploration, interior, combat, or cinematic-lite profiles.

| Function | Purpose |
|---|---|
| CaptureCurrentSettingsToAsset / CaptureCurrentSettingsToAssignedAsset | Write the current component settings into a data asset. |
| ApplyCameraSettingsAsset / ApplyAssignedSettingsAsset | Immediate preset application. Transient runtime state resets deliberately. |
| ApplyCameraSettingsAssetSmooth / ApplyAssignedSettingsAssetSmooth | Blend visible framing values into a new preset while preserving the current visible camera state. |

## What smooth apply blends

- Camera distance
- Pivot height
- Permanent camera offset
- Permanent pivot offset
- Base FOV
- Shoulder side and shoulder offset
- Existing runtime framing layers easing back to zero under the new preset

# 9. Debugging, Testing, and Troubleshooting

## Recommended test passes

- Open-area movement: verify shoulder framing, follow, anticipation, and pitch/speed modifiers.
- Wall and corner test: verify collision compression, recovery, and predictive avoidance.
- Occluder test: place a dither-ready wall between camera and player and verify smooth fade in/out.
- Focus test: focus an actor, release focus, and ensure rotation returns cleanly.
- Preset test: create two deliberately different settings assets and toggle with ApplyCameraSettingsAssetSmooth.
- Shake test: play an impact shake and ensure the handle can stop it cleanly.

## Useful debug utilities

- Slate debug panel via SetSlateDebugPanelVisible / ToggleSlateDebugPanel.
- Draw camera collision debug and camera state debug from component properties.
- Draw yaw constraint debug to visualise the reference and limits.
- Enable dither debug on the occlusion dither component to visualise the pivot-to-camera sweep and tracked blockers.

## Common pitfalls

| Symptom | Likely cause / fix |
|---|---|
| Shoulder swap appears to do nothing | ShoulderOffset is zero, or you are still expecting AdditionalCameraOffset.Y to drive shoulder placement. |
| Dither does not happen | Mesh material does not expose CameraDitherFade or is not wired through DitherTemporalAA into Opacity Mask. |
| Preset change snaps instead of blending | You used ApplyAssignedSettingsAsset instead of ApplyAssignedSettingsAssetSmooth, or BlendTime is zero. |
| Focus target not found | Check focus tag / search rules / line of sight settings or call SetFocusActor directly. |
| Camera clips behind a wall edge | Retune collision radius, minimum distance, and predictive avoidance rather than relying on shoulder offset alone. |

# 10. Blueprint API Reference: UVoraxiaCameraComponent

## Public callable and query nodes

### Voraxia Camera|Setup

| Function | Notes |
|---|---|
| void SetTargetCamera(UCameraComponent* InTargetCamera) | |
| UCameraComponent* GetTargetCamera() const | |

### Voraxia Camera|Input

| Function | Notes |
|---|---|
| void AddYawInput(float Value) | |
| void AddPitchInput(float Value) | |

### Voraxia Camera|Rotation|Yaw Constraints

| Function | Notes |
|---|---|
| void SetYawConstraint( float ReferenceYaw, float MinYawDelta, float MaxYawDelta, float BlendTime = 0.0f ) | Temporary yaw-limiting utility. |
| void SetYawConstraintAroundCurrentView( float MinYawDelta, float MaxYawDelta, float BlendTime = 0.0f ) | Temporary yaw-limiting utility. |
| void ClearYawConstraint(float BlendTime = 0.0f) | Temporary yaw-limiting utility. |
| bool IsYawConstraintActive() const | |
| float GetYawConstraintReferenceYaw() const | |
| float GetYawConstraintMinDelta() const | |
| float GetYawConstraintMaxDelta() const | |

### Voraxia Camera|Framing

| Function | Notes |
|---|---|
| void SetFraming( float NewCameraDistance, float NewPivotHeight, FVector NewCameraOffset, FVector NewPivotOffset, float NewFOVOffset, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr ) | Runtime framing blend function. |
| void ResetFraming(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | Runtime framing blend function. |
| void SetCameraDistance(float NewCameraDistance, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | Runtime framing blend function. |
| void ResetCameraDistance(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void SetPivotHeight(float NewPivotHeight, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void ResetPivotHeight(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void SetCameraOffset(FVector NewRuntimeCameraOffset, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void ResetCameraOffset(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void SetPivotOffset(FVector NewRuntimePivotOffset, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void ResetPivotOffset(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void SetFOVOffset(float NewRuntimeFOVOffset, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void ResetFOVOffset(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| float GetCurrentCameraDistance() const | |
| float GetCurrentPivotHeight() const | |

| Function | Notes |
|---|---|
| FVector GetCurrentCameraOffset() const | |
| FVector GetCurrentPivotOffset() const | |
| float GetCurrentFOV() const | |

## Voraxia Camera|Framing|Shoulder

| Function | Notes |
|---|---|
| void SetUseRightShoulder(bool bNewUseRightShoulder, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | Shoulder camera control. |
| void SetShoulderOffset(float NewShoulderOffset, float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | |
| void SwapCameraShoulder(float BlendTime = 0.25f, UCurveFloat* BlendCurve = nullptr) | Shoulder camera control. |
| bool IsUsingRightShoulder() const | |
| float GetCurrentShoulderOffset() const | |

## Voraxia Camera|Focus

| Function | Notes |
|---|---|
| void SetFocusActor(AActor* NewFocusActor, float BlendTime = 0.35f, FName SocketName = NAME_None) | Runtime focus entry point. |
| void SetFocusComponent(USceneComponent* NewFocusComponent, float BlendTime = 0.35f, FName SocketName = NAME_None) | Runtime focus entry point. |
| void SetFocusWorldLocation(FVector NewFocusWorldLocation, float BlendTime = 0.35f) | Runtime focus entry point. |
| void ClearFocus(float BlendTime = 0.25f) | Runtime focus entry point. |
| bool IsFocusActive() const | |
| float GetCurrentFocusAlpha() const | |
| FVector GetCurrentFocusLocation() const | |
| bool HasFocusTarget() const | |
| AActor* GetCurrentFocusActor() const | |
| USceneComponent* GetCurrentFocusComponent() const | |
| FString GetCurrentFocusTargetName() const | |
| void FocusDefaultTaggedActor(float BlendTime = -1.0f) | |

## Voraxia Camera|Scan

| Function | Notes |
|---|---|
| bool IsCurrentFocusTargetScannable() const | Scanner-facing query helper. |
| FText GetCurrentFocusScanDisplayName() const | Scanner-facing query helper. |
| FText GetCurrentFocusScanSummary() const | Scanner-facing query helper. |
| float GetCurrentFocusScanTimeSeconds() const | Scanner-facing query helper. |
| FString GetCurrentFocusScanCompositionSummary() const | Scanner-facing query helper. |

## Voraxia Camera|Movement Anticipation

| Function | Notes |
|---|---|
| FVector GetCurrentMovementAnticipationOffset() const | |

## Voraxia Camera|Debug

| Function | Notes |
|---|---|
| bool IsCameraCollisionBlocked() const | |
| float GetDesiredDistanceFromPivot() const | |

| Function | Notes |
|---|---|
| float GetEffectiveDistanceFromPivot() const | |
| float GetCurrentCollisionDistanceFromPivot() const | |
| FVector GetLastPivotLocation() const | |
| FVector GetLastDesiredCameraLocation() const | |
| FVector GetLastFinalCameraLocation() const | |
| FRotator GetDesiredCameraRotation() const | |
| FRotator GetSmoothedCameraRotation() const | |
| FString GetCameraDebugSummary() const | |
| void SetSlateDebugPanelVisible(bool bVisible) | Slate debug UI. |
| void ToggleSlateDebugPanel() | Slate debug UI. |
| bool IsSlateDebugPanelVisible() const | |

## Voraxia Camera|Dynamic Modifiers

| Function | Notes |
|---|---|
| float GetCurrentPitchDistanceOffset() const | |
| float GetCurrentPitchFOVOffset() const | |
| float GetCurrentSpeedDistanceOffset() const | |
| float GetCurrentSpeedFOVOffset() const | |
| float GetCurrentDynamicDistanceOffset() const | |
| float GetCurrentDynamicFOVOffset() const | |

## Voraxia Camera|Camera Shake

| Function | Notes |
|---|---|
| FVoraxiaCameraShakeHandle PlayCameraShake( TSubclassOf ShakeClass, float Scale = 1.0f, ECameraShakePlaySpace PlaySpace = ECameraShakePlaySpace::CameraLocal, FRotator UserPlaySpaceRotation = FRotator::ZeroRotator ) | Manual shake control. |
| bool StopCameraShake(FVoraxiaCameraShakeHandle ShakeHandle, bool bImmediately = false) | Manual shake control. |
| void StopAllCameraShakes(bool bImmediately = false) | |
| bool IsCameraShakePlaying(FVoraxiaCameraShakeHandle ShakeHandle) const | |
| int32 GetActiveCameraShakeCount() const | |

## Voraxia Camera|Settings Asset

| Function | Notes |
|---|---|
| bool ApplyCameraSettingsAsset(UVoraxiaCameraSettingsAsset* SettingsAsset) | Settings asset workflow. |
| bool ApplyCameraSettingsAssetSmooth( UVoraxiaCameraSettingsAsset* SettingsAsset, float BlendTime = 0.35f, UCurveFloat* BlendCurve = nullptr ) | Settings asset workflow. |
| bool CaptureCurrentSettingsToAsset(UVoraxiaCameraSettingsAsset* SettingsAsset) | |
| void CaptureCurrentSettingsToAssignedAsset() | |
| void ApplyAssignedSettingsAsset() | |
| bool ApplyAssignedSettingsAssetSmooth( float BlendTime = 0.35f, UCurveFloat* BlendCurve = nullptr ) | |

# 11. Blueprint API Reference: UVoraxiaCameraOcclusionDitherComponent

This companion component has a smaller public API because most of its value lives in authored properties and automatic runtime behaviour.

## Public callable and query nodes

### Voraxia Camera|Occlusion Dither

| Function | Notes |
|---|---|
| void SetCameraComponent(UVoraxiaCameraComponent* InCameraComponent) | |
| UVoraxiaCameraComponent* GetCameraComponent() const | |
| int32 GetTrackedDitheredComponentCount() const | |
| int32 GetActiveOccludingComponentCount() const | |

## Closing notes

VoraxiaCamera is best treated as a reusable gameplay camera foundation. It is authored as a deterministic, C++-first camera stack with Blueprint hooks for runtime orchestration. The component is especially well suited to third-person exploration, traversal, focus/inspection, and future scanning workflows.